

Contrastive deep embedded clustering

Guoshuai Sheng^a, Qianqian Wang^a, Chengquan Pei^b, QuanXue Gao^{a,*}

^a State Key Laboratory of Integrated Services Networks, Xidian University, Xi'an 710071, China

^b School of Artificial Intelligence, Xidian University, Xi'an 710071, China

ARTICLE INFO

Article history:

Received 15 March 2022

Revised 6 September 2022

Accepted 15 September 2022

Available online 21 September 2022

Communicated by Zidong Wang

Keyword:

Deep Clustering

Autoencoder

Contrastive Learning

ABSTRACT

Deep embedded clustering is a popular unsupervised learning method owing to its outstanding performance in data-mining applications. However, existing methods ignore the difficulty in learning discriminative features via clustering due to the lack of supervision, which can be easily obtained in classification tasks. To alleviate this problem, we build a contrastive learning based deep embedded clustering method, i.e., CDEC. Specifically, our model adopts deep auto-encoders to learn a latent discriminative embedded clustering structure. To overcome the problem of lacking label information, the CDEC constructs positive samples and negative samples with the data reconstructed from the data itself and other data, respectively. By maximizing the distance between positive and negative ones, the CDEC can not only obtain the most representative features but also explore the discriminative features. Extensive experiments on several public datasets demonstrate that our method achieves the state-of-the-art clustering effectiveness. Our codes are available at: <https://github.com/guoshuaiS/contrastive-deep>.

© 2022 Elsevier B.V. All rights reserved.

1. Introduction

Clustering is a fundamental tool in unsupervised machine learning and has been widely applied in many fields like pattern recognition and data mining [1]. It aims to group data into multiple clusters so that samples in the same cluster have high similarity with each other. So far, numerous clustering algorithms are proposed to study distance metrics, embedding methods [2], and clustering validation. A simple but effective clustering method is K-means [3], which adopts Euclidean distance for similarity measurement. However, its performance relies on the initial clustering centroids. Besides, it suffers from poor non-linear representation ability and cannot work well on high-dimensional and complex datasets. Another popular clustering method is spectral clustering which converts the clustering task into an eigen-decomposition problem on the affinity matrix with spectral graph theory. Effective as it is, spectral clustering consumes high computational resources. In summary, these methods cannot well handle high-dimensional data.

Recently, the continuous development of deep neural network (DNN) motivates the emergence of deep clustering. For example, Stacked Autoencoder (AE) [4] builds a deep clustering model that

maps data from the original space to a non-linear latent space. With the reconstruction error and self-expression loss, it can well preserve both the local and global subspace structures; JULE [5] presents a recurrent framework that jointly learns the image clusters and data representation. It employs convolutional neural network (CNN) to learn a good image representation for clustering, and the clustering results further work as supervision to improve the representation learning process. Considering the efficiency of traditional clustering methods on low-dimensional data and the powerful representation ability of deep methods, JULE achieves both efficiency and impressive clustering performance. Deep Clustering [6] interactively groups extracted features into different clusters with K-means and uses the cluster assignments as pseudo-labels to guide the representation learning. However, the inaccuracy of the pseudo labels incurs the error between the clustering results and the ground truth, especially when the iteration number is high. Additionally, these deep clustering methods still suffer from the negative impact of lacking supervision. Very recently, contrastive learning has achieved state-of-the-art performance in the unsupervised learning field. Some clustering models based on it have been proposed [7,8]. However, these methods need to construct positive and negative sample pairs through reasonable data augmentation. Since different datasets require various data augmentation methods depending on their specific characteristics, which makes these methods suffer from low generality. Furthermore, introducing data augmentation may result in high computation complexity.

* Corresponding author.

E-mail addresses: agusssheng@foxmail.com (G. Sheng), qianqian174@foxmail.com (Q. Wang), peichengquan@xidian.edu.cn (C. Pei), qxgao@xidian.edu.cn (Q. Gao).

To alleviate the aforementioned problems, we propose a novel Contrastive Deep Embedded Clustering (CDEC) method. The proposed CDEC leverages stacked auto-encoders to obtain the low-dimensional data representations, which is optimized from the perspective of data distribution of latent feature space. In this way, we disentangle latent space by minimizing the KL divergence of the current data distribution and the target distribution, which makes the obtained clusters iteratively become more and more separated. Besides, we propose to construct positive/negative pairs with the original data and the reconstructed data. Then, the contrastive loss together with the reconstruction loss ensures the learned representations retain the discriminative information that is beneficial to clustering. Different from other clustering methods using contrastive loss, our proposed method does not require additional data augmentation, and the contrastive samples are derived from original and reconstructed samples. This makes the construction of contrastive sample pairs more convenient. Extensive experimental results in real-world datasets demonstrate the superiority of CDEC method. The major contributions are summarized as follows:

- We propose a novel Contrastive Deep Embedded Clustering (CDEC) network, which can map samples from the complex original data space into a discriminative latent space and capture the discriminative information between samples from different clusters.
- We design a training loss with the joint constraint of contrastive loss and reconstruction loss to better distinguish confusing samples. Different from methods based on data augmentation, our method constructs positive pairs from the original sample and the reconstructed one, which is proved to produce cluster-friendly embedding features via extensive experiments.
- We conduct extensive experiments in several datasets and compare CDEC with existing clustering methods, of which the results demonstrate the effectiveness of our proposed method.

2. Related Works

2.1. Deep Clustering

Deep learning based clustering shows promising performance owing to the superior capability in representation learning of deep neural networks, which can jointly learn the clustering results and data representation. Existing methods can be mainly categorized into two types, both of which alternately or simultaneously update cluster assignments and network parameters. They usually use inter-class similarities [9,10] as distance metric to update the network with a supervised training method. For example, Deep Clustering [6] iteratively clusters the features with a traditional clustering algorithm and uses the subsequent assignments as supervision to update the weights of the network. DAC [9] defines cosine distance between one-hot vectors of features as the ground-truth similarities and alternately chooses labeled samples to train the convolutional network. Although the aforementioned methods could simultaneously learn latent representations and get cluster assignments, their performance may be degraded by the errors accumulated during the alternation.

To solve the above problem, the latter methods typically train the network by exploiting the mutual information between features of original data and transformed data. For example, IIC [11] first conducts random transformers to form pairs from original images, then obtains clusters by maximizing mutual information between the cluster assignments of each pair. Very recently, PICA [12] uses the strategy of maximizing the global partition confidence of clustering results to learn the most rational data separation from all the possible clustering structures.

2.2. Contrastive Learning

Contrastive Learning is a popular self-supervised learning method, which maximizes the similarity of positive pairs while minimizing the similarity of negative pairs in the feature space. The construction of positive and negative pairs is critical for contrastive learning and significantly influences data representation. SimCLR [13] develops a nonlinear head architecture, which learns visual representations by maximizing similarities of positive pairs and minimizing the similarity of negative pairs. This method uses data augmentation, a random transformation like cropping and color distorting, to design positive and negative pairs. MoCo [14] regards contrastive learning as dictionary look-up and builds a large and consistent dictionary to perform contrastive unsupervised learning. [15] introduces an optimal metric to quantify alignment and uniformity of feature distribution on the hypersphere. Due to the superior representation extraction ability of contrastive learning, some clustering models based on it have been proposed. Contrastive Clustering [7] simultaneously optimizes the contrastive loss on instance-level and cluster-level to learn representations and cluster assignments. [8] introduces a cluster-level contrastive learning manner, combined with the traditional instance-level contrastive loss, and then proposes a multi-granularity contrastive clustering framework to leverage the overall context of the cluster. These clustering methods all require reasonable imagery augmentation to construct contrastive sample pairs, which might require more effort.

3. Contrastive Deep Embedded Clustering

3.1. Network Architecture

Given a dataset of c clusters X , where $X \in R^{n \times d}$ denotes the n data samples of dimension d . We develop a CDEC network including a fully connected stacked denoising encoder, a fully connected stacked denoising decoder, and a clustering network. The network architecture is illustrated in Fig. 1.

Stacked denoising encoder E : our encoder is a m -layer FNN, and each layer is initialized as the encoder part of a denoising auto-encoder [16]. Therefore, the whole model includes m denoising auto-encoders, and the i -th denoising auto-encoder consists of two layers, where the encoder part can be formularized as below:

$$h_i = f_i(\text{Dropout}(u_i); \theta_i), \quad (1)$$

where u_i denotes the input of the i -th denoising auto-encoder; $\text{Dropout}(\cdot)$ [17] is a stochastic mapping to set a portion of the dimensions of u_i as 0; f_i represents the encoder with learnable parameter θ_i and outputs the low-dimensional feature h_i . The decoder part is responsible for converting h_i into reconstructed data. Its mapping can be formularized as below:

$$\hat{y}_i = g_i(h_i), \quad (2)$$

where g_i represents the i -th decoder and outputs the reconstructed sample $\hat{y}_i$ of u_i . After training the i -th denoising auto-encoder, we use h_i as the input data of the next denoising auto-encoder to train it, which is represented as $u_{i+1} = h_i$. Specially, $u_1 = X$. After training all the denoising auto-encoders, we concatenate their encoder parts to form stacked denoising encoder E and combine these learned parameters as its initial parameters $\theta = \{\theta_1, \theta_2, \dots, \theta_m\}$. The stacked denoising encoder maps the d -dimensional input data X to a low-dimensional latent feature Z , which could be represented as $Z = F(\text{Dropout}(X); \theta)$.

Stacked denoising decoder D : the decoder in our network has an opposite architecture to the stacked denoising encoder E , which consists of $(m - 1)$ layers of fully connected neural networks with

the same parameters. The stacked denoising decoder D reconstructs the samples Y from the latent feature Z . Specifically, we suppose $Y = G(Z)$, where G is the corresponding mapping.

Clustering network: Our clustering network has the same structure as the encoder E: its initial parameter is θ that has been learned in the CDEC network, and its mapping function is still expressed as F . The clustering network aims to improve clustering performance in latent space Z , in which we obtain the current data distribution, the self-training target distribution, and cluster centroids $\{\mu_j\}_{j=1}^c$. We optimize the parameter θ by minimizing the KL divergence of the current data distribution and the target data distribution.

3.2. Loss Function

In this subsection, we will introduce the three loss functions used in the training process, which are reconstruction loss, contrastive loss and clustering loss.

3.2.1. Reconstruction Loss

We use the mean square error between input original data and reconstructed data as the reconstruction loss. The above reconstruction loss is defined as

$$\mathcal{L}_{rec} = \|X - Y\|_F^2, \quad (3)$$

3.2.2. Contrastive Loss

To better learn discriminative information of different categories, we apply contrastive learning to maximize agreement of positive pairs and to minimize agreement of negative pairs. Our method regards the original data and reconstructed data of the same sample as positive pairs. Suppose that the input original data $X^T = \{x_1, x_2, \dots, x_n\} \in \mathbb{R}^{d \times n}$ and the output reconstructed data $Y^T = \{y_1, y_2, \dots, y_n\} \in \mathbb{R}^{d \times n}$, and for each sample x_i , we can construct a positive pair with x_i and its corresponding reconstructed sample y_i and $2n - 2$ negative pairs with x_i and the other $2n - 2$ samples apart from x_i and y_i . The similarity between x_i and y_j is defined as

$$s(x_i, y_j) = \frac{(x_i^T)(y_j)}{\|x_i\| \|y_j\|}, \quad (4)$$

where $i, j \in [1, n]$. The similarity between y_j and x_i can be obtained by replacing the positions of x_i and y_j in the above expression. To reduce the similarity between negative pairs at the same time, the loss functions of sample x_i and of sample y_i are denoted as follows

$$\ell_i^x = -\log \frac{\exp(s(x_i, y_i)/\tau)}{\sum_{j=1}^n [\exp(s(x_i, x_j)/\tau) + \exp(s(x_i, y_j)/\tau)]}, \quad (5)$$

$$\ell_i^y = -\log \frac{\exp(s(y_i, x_i)/\tau)}{\sum_{j=1}^n [\exp(s(y_i, y_j)/\tau) + \exp(s(y_i, x_j)/\tau)]}, \quad (6)$$

where τ denotes a temperature parameter to control the degree of attention to hard negative pairs. A suitable temperature coefficient can enable contrastive learning to obtain better representation, and it affects the degree of weight distribution of contrastive loss to different negative samples. A very small value will cause the contrastive loss to only focus on a few negative samples that are closest to the positive samples, and an excessively large value will cause the contrastive loss to pay equal attention to all negative samples. In our experiments, we let $\tau = 0.5$. Finally, the contrastive

loss is computed over each sample of X and Y , and its expression is defined as

$$\mathcal{L}_{con} = \frac{1}{2n} \sum_{i=1}^n (\ell_i^x + \ell_i^y). \quad (7)$$

3.2.3. Clustering Loss

The reconstruction loss and the contrastive loss make our CDEC network generate reconstructed samples which are similar to original samples and enable our latent representations to have a better discriminative ability. To further increase the distance between different classes in subspace and obtain a rational low-dimensional data distribution, we use KL-divergence as the clustering loss to optimize clustering performance. For a specific input sample $x_i \in X^T$, the output latent representation in the clustering network is represented as $z_i = F(x_i; \theta)$. Given the initial cluster centroids $\{\mu_j\}_{j=1}^c$, according to [18], we compute the similarity between latent feature z_i and centroid μ_j by using the Student's t-distribution as a kernel.

$$q_{ij} = \frac{\left(1 + \|z_i - \mu_j\|^2 / \alpha\right)^{-\frac{\alpha+1}{2}}}{\sum_{j'} \left(1 + \|z_i - \mu_{j'}\|^2 / \alpha\right)^{-\frac{\alpha+1}{2}}}, \quad (8)$$

where $i \in [1, n], j \in [1, c]$, α indicates the degrees of freedom of the Student's t-distribution, and q_{ij} denotes the probability of assigning sample x_i to cluster j , which can be also called soft assignment. In our experiments, α is set as 1. The clusters are iteratively refined by learning from their high confidence assignments with the help of an auxiliary target distribution. In our clustering network, we train it by matching the soft assignment to the target distribution. To this end, we use KL divergence between the soft assignment q_{ij} and the auxiliary target distribution p_{ij} as our clustering loss

$$\mathcal{L}_{clu} = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}, \quad (9)$$

In our experiments, the auxiliary distribution is defined as follows

$$p_{ij} = \frac{q_{ij}^2 / f_i}{\sum_{j'} q_{ij'}^2 / f_{j'}}, \quad (10)$$

where $f_j = \sum_i q_{ij}$ are soft cluster frequencies. In the above expression, we first calculate the second power of q_{ij} to emphasize on sample points of higher confidence and then normalize it by frequency per cluster to avoid distorting the latent representations.

3.2.4. Overall Loss

The previous definitions of reconstruction loss and contrastive loss are applied to the entire stacked denoising auto-encoder in the formal training stage. In the pre-training stage, we can get the corresponding losses of training the i -th denoising auto-encoder by replacing X and Y in Eq. (3), (7) with u_i and $\hat{y}_i$. Therefore, the objective functions of the two stages are expressed as follows

$$\mathcal{L}_{pre} = \mathcal{L}_{rec} + \alpha \mathcal{L}_{con}, \quad (11)$$

$$\mathcal{L}_{all} = \mathcal{L}_{rec} + \beta \mathcal{L}_{con} + \mathcal{L}_{clu}. \quad (12)$$

where $\mathcal{L}_{pre}$ denotes the pre-training loss, $\mathcal{L}_{all}$ denotes the overall loss, α is a coefficient to control the impact of contrastive loss in the pre-training phase and β plays the same role in the formal training phase.

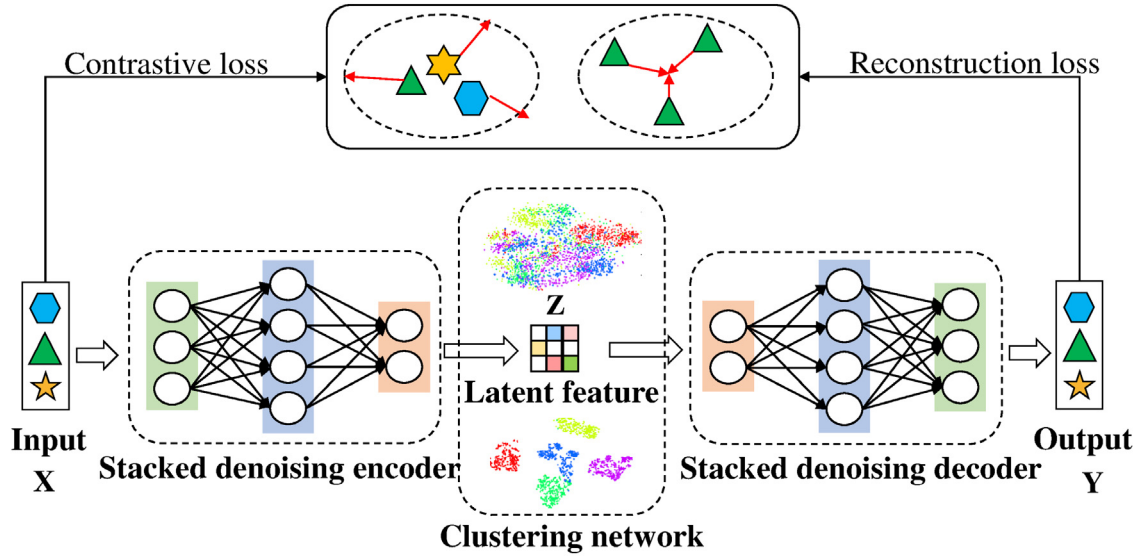


Fig. 1. The structure of CDEC network. It consists of three components: Stacked denoising encoder, Stacked denoising decoder, and clustering network. The encoder network outputs a low-dimensional latent feature Z , the decoder network generates the reconstructed samples Y . The clustering network optimizes the KL divergence between the original distribution with the target distribution for better clustering performance.

3.3. Training Procedure

The training procedure is composed of three steps. The pseudo-code of CDEC is illustrated in Algorithm 1. The specific training process is described in the following.

Algorithm 1: CDEC algorithm

Input: dataset X ; pre-training epochs n_1 ; formal training epochs n_2 ; cluster number c ; denoising encoder number m .
Output: cluster assignments.

// Pre-training stage:

```

1: Set  $u_1 = X$ ;
2: for  $i = 1$  to  $m$  do
3:   fore  $epoch = 1$  to  $n_1$  do
4:      $h_i = f_i(\text{Dropout}(u_i); \theta_i)$ ;  $\hat{y}_i = g_i(h_i)$ ; Compute pre-
       training loss  $\mathcal{L}_{pre}$  by Eq. 11; Update  $f_i, g_i$  to obtain  $\theta_i$ ;
5:   end for
6:    $u_{i+1} = h_i$ ;
7: end for
8:  $\theta = \{\theta_1, \theta_2, \dots, \theta_m\}$ ;

```

// Formal training stage

```

1:  $Z = F(X; \theta)$ ; Perform the K-means algorithm on  $Z$ ; Obtain
   the initial clustering centroids  $\{\mu_j\}_{j=1}^c$ ;
2: fore  $epoch = 1$  to  $n_2$  do
3:    $Z = F(\text{Dropout}(X); \theta)$ ;  $Y = G(Z)$ ; Compute formal training
       loss  $\mathcal{L}_{all}$  by Eq. 12; Update  $F, G$  and  $\{\mu_j\}_{j=1}^c$ ;

```

4: **endfor**

// Testing stage

```

1:  $Z = F(X; \theta)$ ; Conduct the K-means algorithm on  $Z$  to obtain
   cluster assignments;

```

Step 1: Training each denoising auto-encoder to obtain the initial parameter θ of stacked denoising encoder E , which is also named the pre-training stage. Specifically, we take u_i as the input of the i -th denoising auto-encoder and get its output $\hat{y}_i$. Then we iteratively minimize $\mathcal{L}_{pre}$ to update f_i, g_i and save encoder mapping parameter θ_i . After training one denoising auto-encoder, we use its encoded output

h_i as the input of the next layer. After training all layers, we concatenate all encoder parts of each auto-encoder to form encoder E of CDEC network, build decoder D in reverse training order, and save initial parameter θ for the next step.

Step 2: Training CDEC by minimizing the sum of reconstruction loss, contrastive loss, and clustering loss, i.e., $\mathcal{L}_{all}$ to update encoder E , decoder D and clustering centroids. This step is also named the formal training stage. The third loss requires initial clustering centroids to work, so we first perform K-means algorithm on latent feature Z to get it, i.e., $\{\mu_j\}_{j=1}^c$. Then we compute $\mathcal{L}_{all}$ to update the network parameters.

Step 3: Testing stage. After the first two steps are finished, we perform the K-means algorithm on Z and get our experiment result.

4. Experimental results

4.1. Experimental Settings

Datasets and Description: In order to validate the performance of the proposed method, we perform an experiment on six benchmark datasets. A brief description of these datasets is given as follows. (1) IAPR-TC12 (IAPR) [19]: This database contains 20000 images with 6 clusters, and each image is paired by a short text introduction. 7855 images without text descriptions, grouped into 6 clusters, were randomly selected in our experiment. (2) ESP-Game (ESP) [20]: It is a collection of social pictures searched from a picture tagging website, consisting of 20770 pictures and corresponding tags. We only select 11,032 images with approximately 5 tags per image. (3) MIRFlickr (Flickr) [21]: It is an image dataset containing 25000 samples with 1386 tags that include some duplicate and polluted pictures. So we denoise and de-duplicate these data to construct a dataset with 12,154 images with 6 classes. (4) MNIST [22]: It consists of 70000 handwritten digits from 0 to 9, which are divided into a training set and a test set. In our experiment, only the training set of 60000 handwritten digits is used. (5) USPS [23]: It is a digit dataset containing a total of 1854 grayscale samples of 16×16 pixel size of 10 classes; (6) COIL20 [24]: It

consists of 1440 grayscale images of 128×128 pixel size of 20 classes. We convert the size of these images into 32×32 to adapt our model. Detailed information of six datasets are summarized in Table 1.

Comparison Baselines: We employ six state-of-the-art clustering algorithms for comparison to evaluate the superiority of the proposed algorithm. (1) K-means clustering [3] is a traditional method that is usually used to partition a data set into k categories. (2) Auto-encoder (AE) [25] is a neural network that contains an encoder to reduce the dimensionality of the input data and a decoder to reconstruct the input data. We perform K-means clustering in its coding space. (3) Stacked denoising auto-encoder (SDAE) [16] can learn useful representations in a deep network with a local denoising criterion. We perform K-means clustering in its subspace representations. (4) Deep Embedded Clustering (DEC) [18] makes use of a fully connected stacked denoising auto-encoder network to learn the latent representations by minimizing the reconstruction loss in the pre-training phase. The Kullback Leibler divergence based on a target distribution is the objective function to improve clustering performance in the clustering phase. (5) A Robust and Stable K-means Variant (t-k-means) [26] is a robust clustering method that is derived from the t-mixture model, in which data is sampled from the t-mixture distribution. (6) Contrastive Clustering (CC) [7] is a deep clustering method based on contrastive learning, which can simultaneously obtain representation and clustering assignments. Its network structure consists of three parts: pair construction backbone, instance-level contrastive head, and cluster-level contrastive head. For a fair comparison, we replace the backbone with the encoder network E in our experiments.

Evaluation Metrics: To evaluate the clustering performance of different methods, clustering accuracy (ACC) and normalized

mutual information (NMI) are commonly applied [27]. A higher value indicates better clustering performance for all the metrics. We run each experiment 10 times repeatedly and report the average performance in the experiments.

Experimental Setup: In the proposed CDEC method, we set the encoder dimensions to $\{d, d/2, d/2, 2d, c\}$ and set the decoder dimensions to $\{2d, d/2, d/2, d\}$ for all datasets, where d is the dimension of the original data and c is the clusters. All layers of the network are fully connected, which is followed by a ReLU layer to perform data non-linear activation. We train each layer as a denoising auto-encoder separately for 150 epochs with a dropout rate of 0.2 and load these learned parameters into our CDEC network to initialize it. During the formal training stage, we conduct 200 epochs to train the network with a dropout rate of 0.2. Then the clustering network is initialized with the encoder part of our CDEC network, and the KL divergence serves as an objective function to train it. In terms of parameters setting, both α and β are set as 0.3, and the minibatch size is set as 256 for all training phases of the network. We use the Adam [28] optimizer with a learning rate of 0.005. Our proposed CDEC method and other deep clustering methods are implemented under PyTorch-1.7.0. Other traditional clustering baselines are conducted on Matlab 2017a. We run all codes on the Windows 10 platform with NVIDIA 1080 Graphics Processing Units (GPUs) and 32 GB memory.

4.2. Experimental Results

In this subsection, the comparison of different methods on six challenging datasets is presented. Particularly, several meaningful observations and discussions could be made as follows.

Comparison Results: The clustering performances of CDEC and other state-of-the-art baselines on six datasets are reported in Table 2 and Table 3. According to the presented results, we get the following meaningful discussions: (1) On all six datasets, the proposed CDEC method significantly outperforms other methods by a clear improvement. In particular, CDEC surpasses DEC by 8.88% on MNIST in terms of ACC. The promising results indicate the effective clustering ability of CDEC, which benefits from the incorporation of contrastive learning and the KL divergence optimization. This indicates that contrastive loss can both learn the dif-

Table 1
Statistics of the six datasets.

Dataset	IAPR	ESP	Flickr	MNIST	USPS	COIL20
Clusters	6	7	6	10	10	20
Size	7855	11032	12154	60000	1854	1440

Table 2
Clustering performance of different methods on the first three datasets. The best results are highlighted in bold.

Methods	IAPR		ESP		Flickr	
	ACC	NMI	ACC	NMI	ACC	NMI
K-means	0.281±0.006	0.073±0.009	0.276±0.004	0.124±0.006	0.248±0.009	0.056±0.007
AE	0.415±0.005	0.191±0.007	0.369±0.005	0.194±0.007	0.413±0.006	0.218±0.004
SDAE	0.430±0.004	0.198±0.005	0.372±0.004	0.199±0.007	0.491±0.006	0.261±0.006
DEC	0.443±0.004	0.201±0.005	0.393±0.005	0.214±0.006	0.519±0.005	0.283±0.006
t-k-means	0.417±0.003	0.180±0.004	0.361±0.004	0.205±0.003	0.409±0.008	0.223±0.005
CC	0.448±0.006	0.216±0.007	0.396±0.005	0.219±0.006	0.523±0.008	0.291±0.007
CDEC	0.456±0.005	0.230±0.004	0.412±0.005	0.235±0.006	0.532±0.006	0.315±0.006

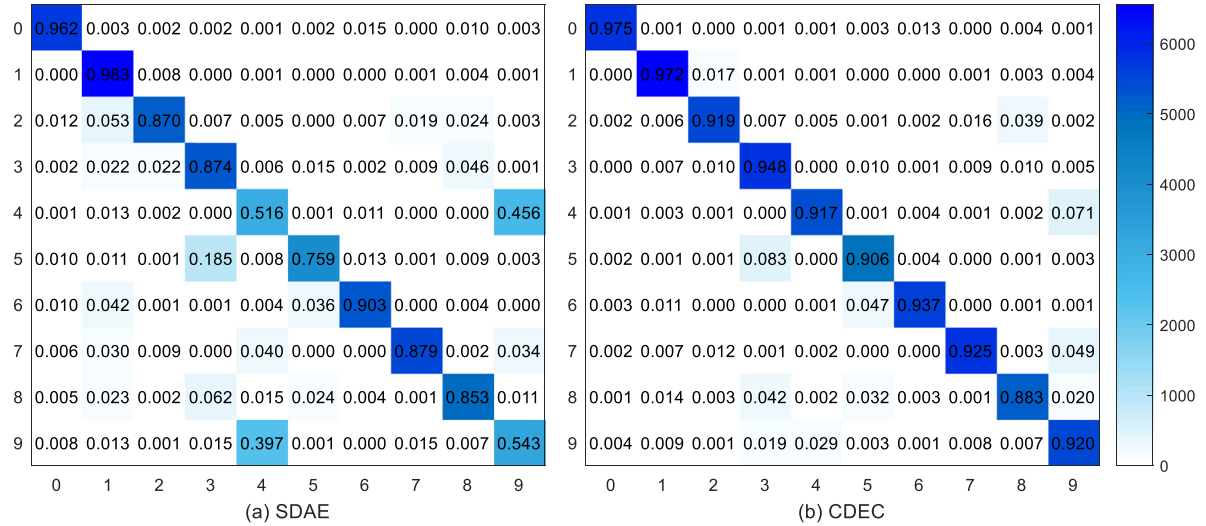
Table 3
Clustering performance of different methods on the last three datasets. The best results are highlighted in bold.

Methods	MNIST		USPS		COIL20	
	ACC	NMI	ACC	NMI	ACC	NMI
K-means	0.549±0.011	0.487±0.014	0.649±0.015	0.627±0.013	0.678±0.022	0.769±0.015
AE	0.616±0.008	0.509±0.013	0.720±0.013	0.694±0.011	0.742±0.018	0.805±0.010
SDAE	0.818±0.007	0.747±0.010	0.740±0.010	0.709±0.010	0.751±0.009	0.837±0.009
DEC	0.844±0.008	0.797±0.009	0.768±0.010	0.721±0.008	0.761±0.010	0.841±0.009
t-k-means	0.620±0.009	0.536±0.012	0.679±0.009	0.618±0.008	0.633±0.009	0.731±0.009
CC	0.886±0.007	0.820±0.010	0.806±0.011	0.748±0.009	0.784±0.007	0.859±0.006
CDEC	0.933±0.007	0.858±0.009	0.772±0.007	0.729±0.007	0.790±0.009	0.861±0.008

Table 4

The ablation study on the first four challenging datasets under the metrics of ACC and NMI. Different methods use constraint losses identified by ✓. Best results are highlighted in bold.

Methods	Reconstruction Loss	Contrastive Loss	Clustering Loss	IAPR		ESP		Flickr		MNIST	
				ACC	NMI	ACC	NMI	ACC	NMI	ACC	NMI
AE	✓			0.415	0.191	0.369	0.194	0.413	0.218	0.616	0.509
Method-A		✓		0.328	0.128	0.316	0.150	0.346	0.128	0.550	0.460
CAE	✓	✓		0.433	0.193	0.377	0.188	0.431	0.230	0.631	0.531
SDAE	✓			0.430	0.198	0.372	0.199	0.491	0.261	0.818	0.747
Method-C		✓		0.426	0.195	0.382	0.202	0.474	0.259	0.753	0.651
CSDAE	✓	✓		0.434	0.205	0.383	0.211	0.493	0.270	0.894	0.794
CDEC	✓	✓	✓	0.456	0.230	0.412	0.235	0.532	0.315	0.933	0.858

**Fig. 2.** The illustration of confusion matrix of the subspace feature matrices on MNIST.

ference information and the category information between original data and reconstructed data. (2) The CDEC method has superior performance to traditional linear methods, which indicates that the CDEC method can more effectively capture nonlinear representations. In light of this, the network in CDEC has a higher discriminative ability in the clustering task. (3) Although our method does not perform as well as CC on the USPS dataset, it does not require complex data pairs construction. On different datasets, CC requires different ways of constructing contrastive sample pairs to achieve higher accuracy. Our contrastive samples are derived from raw and decoded data. Therefore, compared with other contrastive-learning-based methods, our proposed method can balance the accuracy and the convenience of contrastive sample construction.

Ablation Study: To analyze the role of contrastive loss in the network, we perform an ablation study on the first four challenging datasets. Specifically, we conduct experiments in the auto-encoder network and the stacked denoising auto-encoder network as follows: (1) Only with reconstruction loss (AE and SDAE). In this scenario, we only use the reconstruction loss as the objective function. (2) Only with contrastive loss (Method-A and Method-C). In this setting, we replace the reconstruction loss with the contrastive loss as our objective function. (3) With reconstruction loss and contrastive loss (CAE and CSDAE). (4) With overall loss (CDEC). The result of our ablation study is reported in Table 4. Both the comparison with AE, CAE, SDAE, and CSDAE confirm that the combination of these two losses is effective. To further demonstrate the advantages of the proposed method over the overall objective loss, we conduct the experiment of confusion matrix of the subspace feature matrices on MNIST. The result is shown in Fig. 2. In the confusion matrix, the diagonal elements represent the clustering accu-

racy for different categories. On the diagonal, the deeper color indicates that the clustering performance is better. Fig. 2 shows that the addition of contrastive loss makes it easier for the network to distinguish confusing samples. To further evaluate the impact of contrastive loss on image reconstruction quality, we compare the average per-sample reconstruction error between DEC and CDEC on six datasets. The result is shown in Table 5. Table 5 shows that the reconstruction error of CDEC is smaller than that of DEC. The addition of contrastive loss makes decoded samples not only as similar as possible to their original samples but also as dissimilar to the negative samples as possible. Therefore, the autoencoder has the ability to capture the global information of samples.

Analysis of Subspace feature: In this subsection, we study the impact of different methods on the data distribution of the corresponding subspace features. In order to show a t-SNE [29] visualization for feature embeddings in terms of different methods on the MNIST dataset, Fig. 3 is given, in which different colors indicated different labels. The figure shows that our proposed CDEC method has a more distinct and clear data distribution than all the other methods. This clearly confirms that the KL divergence optimization can improve the rationality of cluster assignments and that our objective loss makes the network more discriminative for different types of samples with high similarity.

Table 5

The reconstruction error between DEC and CDEC on six datasets.

Methods	IAPR	ESP	Flickr	MNIST	USPS	COIL20
DEC	14.492	39.965	13.712	31.172	20.243	48.165
CDEC	12.544	30.450	12.574	21.232	18.817	46.300

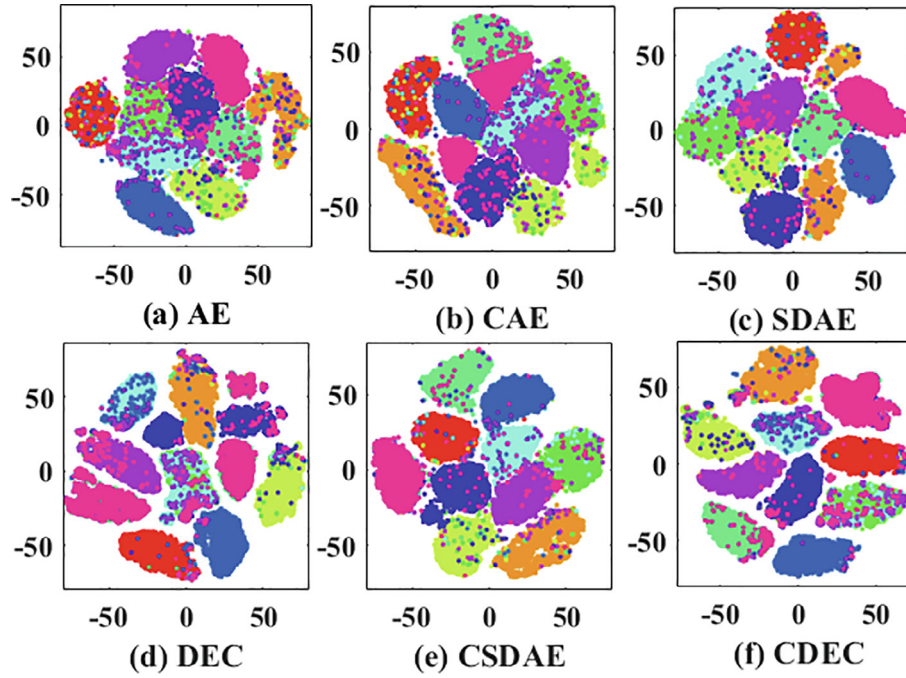


Fig. 3. Visualization of the subspace features given by different methods with t-SNE on MNIST, where (a) AE, (b) CAE, (c) SDAE, (d) DEC, (e) CSDAE, and (f) CDEC.

Impact of Different Parameters: To research the effectiveness of parameters, we employ experiments of controlling variables in our CDEC. In detail, we keep α unchanged in the phase of training each layer of denoising auto-encoders, then we change β in the phase of training the overall deep network and observe the ACC values on the MNIST dataset. As shown in Fig. 4, the ACC of CDEC has an increment with the gradual raising of β , which indicates that

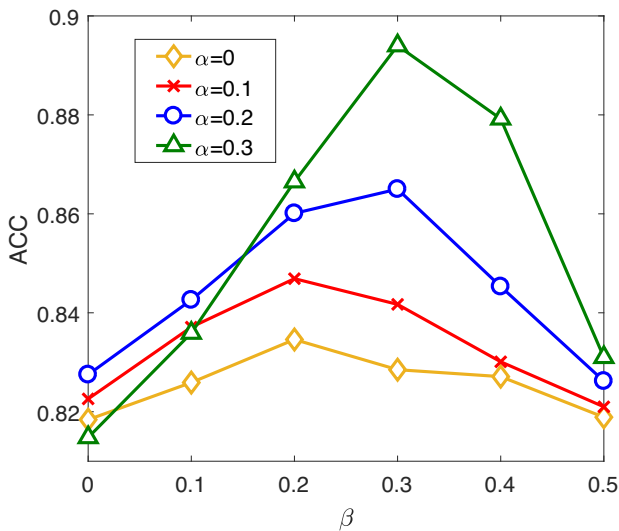


Fig. 4. The impact of parameters on MNIST. When α is set as different values (0, 0.1, 0.2, 0.3, 0.4, 0.5), we keep α unchanged in each line, and observe the change of ACC with β .

the clustering performance of our network is gradually increasing. This is because the incorporation of a contrastive loss forces the network to distinguish between positive and negative samples [13]. But as β increases further, the clustering accuracy declines to some extent. It is worth noting that when both α and β are set as 0.3, the CSDAE method has the best ACC value. The reason for this phenomenon is that too large a contrastive constraint reduces the reconstruction performance, thereby destroying latent representations. An extreme case is that some different samples of the same label are classified into different labels. This can also be verified by the results of Method-A and Method-C in our ablation study.

Time Complexity Comparison: In this subsection, to evaluate the time complexity, we report the running time in seconds of different clustering methods on the MNIST dataset, as shown in Table 6. K-means and t-k-means are classical methods, which consume less time. We test the running time of other deep clustering methods for 200 epochs for fairness. We can see that, compared with other methods, the computation complexity of the proposed model is reasonable.

5. Conclusion

In this paper, we propose a Contrastive Deep Embedded Clustering, which maps original data space into a discriminative and low-dimensional representation space. By designing a contrastive head, we introduce contrastive loss to the deep auto-encoder without performing data augmentation. Thus, the data features from the same cluster are aggregated. To disentangle clusters, we minimize the KL divergence to obtain a more plausible clustering structure and improve cluster assignments. Experiments on several public datasets show its promising performance. In the future,

Table 6

The running time of all methods on MNIST dataset.

Methods	K-means	AE	SDAE	DEC	t-k-means	CC	CDEC
Running time(s)	53	8154	13919	14245	1789	19126	15978

we plan to extend it to cross-modal clustering and semantic segmentation.

CRedit authorship contribution statement

Guoshuai Sheng: Software, Writing - original draft. **Qianqian Wang:** Methodology, Writing - review & editing. **Chengquan Pei:** Writing - review & editing. **Quanxue Gao:** Conceptualization.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work is supported by National Natural Science Foundation of China under Grant 62176203, 62101302, Initiative Postdocs Supporting Program, China Postdoctoral Science Foundation (Grant 2019M653564) and the Open Projects Program of National Laboratory of Pattern Recognition.

References

- [1] J.V. Munoz, M.A. Gonçalves, Z. Dias, R.D.S. Torres, Hierarchical clustering-based graphs for large scale approximate nearest neighbor search, *PR* 96 (2019) 106970.
- [2] D. Wu, F. Nie, X. Dong, R. Wang, X. Li, Parameter-free consensus embedding learning for multiview graph-based clustering, *IEEE TNNLS*.
- [3] J.A. Hartigan, M.A. Wong, Algorithm as 136: A k-means clustering algorithm, *Journal of the royal statistical society. series c (applied statistics)* 28 (1) (1979) 100–108.
- [4] X. Peng, J. Feng, S. Xiao, W.-Y. Yau, J.T. Zhou, S. Yang, Structured autoencoders for subspace clustering, *IEEE TIP* 27 (10) (2018) 5076–5086.
- [5] J. Yang, D. Parikh, D. Batra, Joint unsupervised learning of deep representations and image clusters, in: *IEEE CVPR*, 2016, pp. 5147–5156.
- [6] M. Caron, P. Bojanowski, A. Joulin, M. Douze, Deep clustering for unsupervised learning of visual features, in: *ECCV*, 2018, pp. 132–149.
- [7] Y. Li, P. Hu, Z. Liu, D. Peng, J.T. Zhou, X. Peng, Contrastive clustering, in: 2021 AAAI Conference on Artificial Intelligence (AAAI), 2021.
- [8] Y. Shen, Z. Shen, M. Wang, J. Qin, P. Torr, L. Shao, You never cluster alone, *Advances in Neural Information Processing Systems* 34.
- [9] J. Chang, L. Wang, G. Meng, S. Xiang, C. Pan, Deep adaptive image clustering, in: *IEEE ICCV*, 2017, pp. 5879–5887.
- [10] J. Wu, K. Long, F. Wang, C. Qian, C. Li, Z. Lin, H. Zha, Deep comprehensive correlation mining for image clustering, in: *IEEE ICCV*, 2019, pp. 8150–8159.
- [11] X. Ji, J.F. Henriques, A. Vedaldi, Invariant information clustering for unsupervised image classification and segmentation, in: *IEEE ICCV*, 2019, pp. 9865–9874.
- [12] J. Huang, S. Gong, X. Zhu, Deep semantic clustering by partition confidence maximisation, in: *IEEE CVPR*, 2020, pp. 8849–8858.
- [13] T. Chen, S. Kornblith, M. Norouzi, G. Hinton, A simple framework for contrastive learning of visual representations, in: *ICML*, 2020, pp. 1597–1607.
- [14] K. He, H. Fan, Y. Wu, S. Xie, R. Girshick, Momentum contrast for unsupervised visual representation learning, in: *IEEE CVPR*, 2020, pp. 9729–9738.
- [15] T. Wang, P. Isola, Understanding contrastive representation learning through alignment and uniformity on the hypersphere, in: *ICML*, 2020, pp. 9929–9939.
- [16] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, P.-A. Manzagol, L. Bottou, Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion., *Journal of machine learning research* 11 (12).
- [17] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, Dropout: a simple way to prevent neural networks from overfitting, *The Journal of Machine Learning Research* 15 (1) (2014) 1929–1958.
- [18] J. Xie, R. Girshick, A. Farhadi, Unsupervised deep embedding for clustering analysis, in: *ICML*, 2016, pp. 478–487.
- [19] M. Grubinger, P. Clough, H. Müller, T. Deselaers, The iapr tc-12 benchmark: A new evaluation resource for visual information systems, in: *International workshop ontolmage*, Vol. 2, 2006.
- [20] L. Von Ahn, L. Dabbish, Labeling images with a computer game, in: *SIGCHI*, 2004, pp. 319–326.
- [21] M.J. Huiskes, M.S. Lew, The mir flickr retrieval evaluation, in: *ACM ICIR*, 2008, pp. 39–43.
- [22] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning applied to document recognition, *Proceedings of the IEEE* 86 (11) (1998) 2278–2324.
- [23] J.J. Hull, A database for handwritten text recognition research, *IEEE TPAMI* 16 (5) (1994) 550–554.
- [24] F. Li, H. Qiao, B. Zhang, Discriminatively boosted image clustering with fully convolutional auto-encoders, *PR* 83 (2018) 161–173.
- [25] Y. Bengio, P. Lamblin, D. Popovici, H. Larochelle, Greedy layer-wise training of deep networks, in: *NeurIPS*, 2007, pp. 153–160.
- [26] Y. Li, Y. Zhang, Q. Tang, W. Huang, Y. Jiang, S.-T. Xia, t-k-means: A robust and stable k-means variant, in: *ICASSP*, 2021, pp. 3120–3124.
- [27] W. Xu, X. Liu, Y. Gong, Document clustering based on non-negative matrix factorization, in: *ACM SIGIR*, 2003, pp. 267–273.
- [28] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, *arXiv preprint arXiv:1412.6980*.
- [29] L. Van der Maaten, G. Hinton, Visualizing data using t-sne., *Journal of machine learning research* 9 (11).



Guoshuai Sheng received the B. Eng. degree in Electronic and Information Engineering from Xi'an University of Technology, China, in 2021. He is currently working toward the M.S. degree in Communication and Information Systems from Xidian University, China. His research interests include deep clustering and self-supervised learning.



Qianqian Wang received the B.Eng. degree in communication engineering from Lanzhou University of Technology, China, in 2014, the Ph.D. degree from Xidian University, Xi'an China, in 2019. She was a visiting scholar with Northeastern University, U.S., from October 2017 to October 2018. She is currently a lecturer with the School of Telecommunications Engineering, Xidian University, China. Her research interests include multi-view learning, pattern recognition, dimensionality reduction, sparse representation, and face recognition.



Chengquan Pei received the B.Eng. degree in communication engineering from Lanzhou University of Technology, China, in 2010, the Ph.D. degree from Xi'an Jiaotong University, Xi'an China, in 2017. He was a post doctor in Tsinghua university from October 2018 to October 2021. He is currently a lecturer with the School of Artificial Intelligence, Xidian University, China. His research interests include computational photography, circuit and system, and computational femtosecond imaging.



machine learning

Quanxue Gao received the B.Eng. degree from Xi'an Highway University, Xi'an, China, in 1998, the M.S. degree from the Gansu University of Technology, Lanzhou, China, in 2001, and the Ph.D. degree from Northwestern Polytechnical University, Xi'an China, in 2005. He was a associate research with the Biometrics Center, The Hong Kong Polytechnic University, Hong Kong from 2006 to 2007. He is currently a professor with the School of Telecommunications Engineering, Xidian University, and also a key member of State Key Laboratory of Integrated Services Networks. His current research interests include pattern recognition and